

Every endpoint, scope & server function of the Emma Dashboard.

How the dashboard talks to its upstream analytics API, how the six key scopes gate what a workspace can see and do, and the server-side function layer that keeps the agency key and every client's data isolated.

DOCUMENT

API Reference v1

DATE

July 2026

AUDIENCE

Engineering · internal

PRODUCT

Emma Dashboard

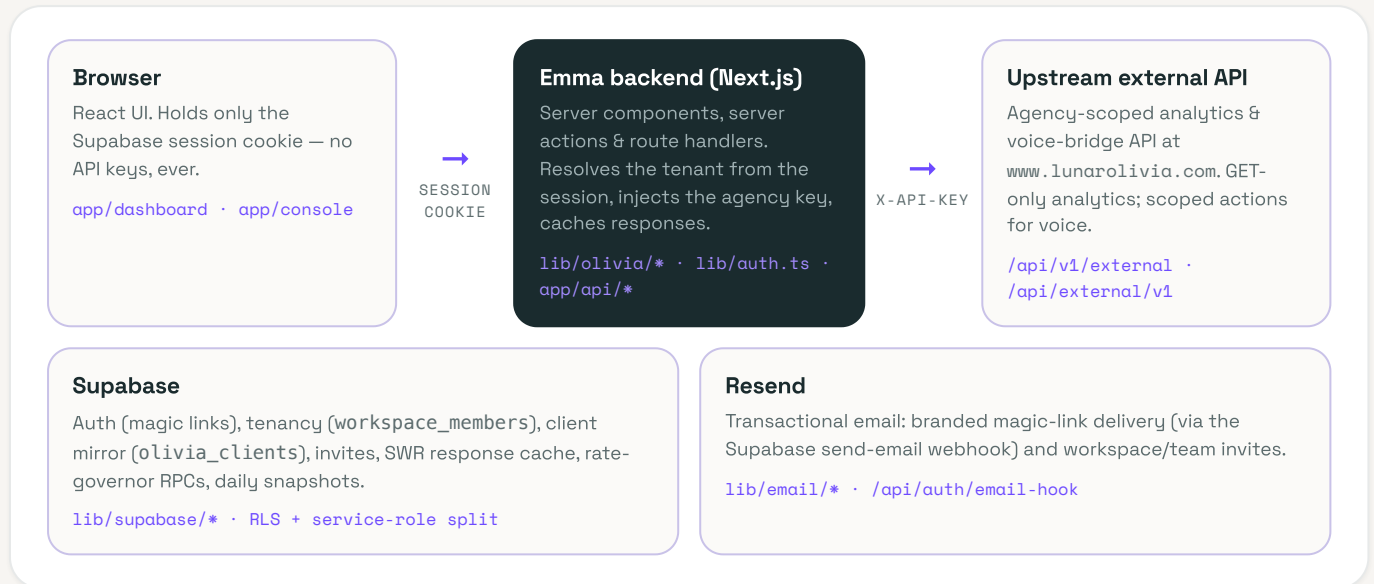
CONTENTS

01	Architecture at a glance	layers & isolation	06	Dashboard HTTP endpoints	same-origin proxies
02	Authentication & scopes	API key · user roles	07	Server function layer	lib/olivia · actions
03	Conventions	params · errors · limits	08	Enums & metric definitions	exact semantics
04	Analytics API	discovery + 16 endpoints	09	Environment & security	secrets · checklist
05	Voice bridges	briefing · reporting			

Naming. User-facing copy in the product always says **Emma**. The upstream vendor platform appears in this document only through its technical identifiers (hostnames, env vars, lib/olivia/) because those are the literal values engineering works with.

Architecture at a glance

Emma is a read-only, multi-tenant analytics dashboard. Three layers, one rule: the browser never holds a key, and every data fetch is scoped to a `client_id` derived from the signed-in session — never from the browser.



1.1 Isolation guarantees

- **One key, one agency.** The dashboard holds a single agency-scoped API key. The key *is* the agency — no `agency_id` is ever passed; a foreign `client_id` returns `404 client_not_found`, never data.
- **Session-derived tenant.** `getSessionClientId()` (`lib/auth.ts`) is the single source of tenant identity. Members are locked to exactly one client; the `emma-active-client` switching cookie is honored only for platform admins, and only after validation against the agency's own client set.
- **Server-only key.** All upstream modules are `server-only`. The SSE and recording proxies exist precisely so the browser can consume cross-origin streams without ever seeing the key.
- **Tenant-namespaced cache.** Every cache key includes `client_id` — isolation by construction, not just an optimization.

Authentication & scopes

Two independent auth systems: an **agency API key** (server-to-server, upstream) and **Supabase user sessions** (browser, dashboard). Scopes gate the key; roles gate the user.

2.1 The agency API key

Every upstream request sends the key in a header. It is minted by an upstream operator (Settings → API Keys) and lives only in the `OLIVIA_API_KEY` env var on Emma’s backend.

```
# every server-to-server request
x-api-key: oa_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

2.2 Key scopes

SCOPE	GRANTS	WITHOUT IT
● <code>clients:read</code>	Discovering the agency’s clients (§4.1) — the <code>client_id</code> s every other call needs.	403 forbidden_scope
● <code>dashboard:read</code>	All per-client analytics endpoints (§4.2–4.4): overview, timeseries, outcomes, funnel, agents, campaigns, pipelines, calendar, leads, calls, conversations, DM threads, costs.	403 forbidden_scope
● <code>dashboard:pii</code>	Un-redacts PII on list & drill-down endpoints: names, phones, emails, Instagram handles, recording URLs, transcripts, conversation summaries and DM message bodies.	Fields are omitted entirely; conversation threads return <code>locked: true</code> with empty messages.
● <code>dashboard:notes</code>	The one analytics write: <code>PUT .../leads/{leadId}/notes</code> (≤ 20,000 chars).	Notes render read-only in the lead page.
● <code>dashboard:brief</code>	The briefing bridge (§5.1) — starting, polling and ending live voice briefings.	“Brief Emma” degrades to a local simulated preview.
● <code>dashboard:report</code>	The reporting bridge (§5.2) — voice reporting walkthrough sessions (+ SSE transcript). Add <code>dashboard:pii</code> for Emma to speak lead names.	“Reporting walkthrough” degrades to a local simulated preview.

Activation state. The voice bridges are additionally flag-gated in Emma (`OLIVIA_BRIEFING_ENABLED` / `OLIVIA_REPORTING_ENABLED`). Going live requires both the flag *and* the corresponding scope on the production key — the scope grants are the outstanding step at the time of writing.

2.3 Dashboard user auth (Supabase)

SIGN-IN FLOW

- Passwordless **magic link** (`sendMagicLink` , never auto-creates users) with a password fallback.
- Emails are branded & delivered via Resend through the signed `/api/auth/email-hook` webhook.
- The link lands on `/auth/callback` , which exchanges the code (PKCE or `token_hash`) for a session and redirects to `/dashboard` .
- `middleware.ts` refreshes the session on every page navigation and bounces unauthenticated visitors to `/login` . Public paths: `/` , `/login` , `/auth` , `/invite` .

ROLES

- `member` Locked to exactly one client workspace via `workspace_members` . The client-switch cookie is ignored entirely.
- `platform_admin` May switch across all agency clients (validated cookie) and holds the `/console` agency-admin app: invites, team, roster sync, costs.
- `requireAdmin()` gates every console page, console action and every `lib/olivia/agency.ts` export — 403 for members.
- Invited users join via single-use, 7-day tokens (`acceptInvite`) — the token is the credential; conflicting mappings are refused, never silently re-mapped.

Conventions

Shared rules for every upstream endpoint: hosts, parameters, envelopes, errors, limits and PII redaction.

3.1 Base URLs

PURPOSE	BASE PATH
Client discovery	<code>https://www.lunarolivia.com/api/v1/external</code>
Per-client analytics + voice bridges	<code>https://www.lunarolivia.com/api/external/v1</code>

Careful: the segment order differs (`v1/external` vs `external/v1`) — they are different trees. The apex domain 307-redirects to `www`. Host override: `OLIVIA_API_BASE`.

3.2 Query parameters

PARAM	APPLIES TO	DEFAULT	RULES
<code>from</code> , <code>to</code>	analytics endpoints	last 30 days	<code>YYYY-MM-DD</code> , <code>from ≤ to</code> , window ≤ 366 days. Inclusive UTC day bounds; day-buckets use <code>tz</code> .
<code>tz</code>	analytics endpoints	agency tz, else <code>America/New_York</code>	IANA name, e.g. <code>America/Chicago</code> .
<code>page</code>	list endpoints	1	integer ≥ 1 .
<code>limit</code>	list endpoints	25	integer 1–100 (clamped).
<code>status</code> , <code>source</code> , <code>stage_id</code>	<code>/leads</code> only	—	optional exact-match filters.
<code>lead_id</code>	<code>/dm-threads</code> only	—	filter threads to one lead.
<code>month</code> , <code>upcoming_limit</code>	<code>/calendar</code> only	current month	<code>YYYY-MM</code> ; dates returned already client-tz local.

3.3 Envelopes & errors

Detail/aggregate endpoints return an object directly. List endpoints return `{ "<resource>": [...], "total": N, "page": P, "limit": L }`. Errors always share one envelope:

```
{ "error": "Client not found", "code": "client_not_found" }
```

STATUS	CODE	MEANING
401	unauthorized	missing / invalid / revoked / expired key.
403	forbidden_scope	key lacks the required scope (§2.2).
404	client_not_found · lead_not_found · conversation_not_found · agency_not_found · reporting_not_found	id doesn't exist <i>or isn't in this agency</i> — foreign ids are indistinguishable from missing ones.
400	invalid_request · invalid_date_range · invalid_timezone · date_range_too_large	bad from / to / tz or body.
429	rate_limited · reporting_concurrency_limit	back off using the <code>Retry-After</code> header.
500	internal_error	server error — safe to retry with backoff.

In Emma's code these become a typed `OliviaError { status, code, message, retryAfterSeconds? }` (`lib/olivia/errors.ts`).

3.4 Rate limits & PII redaction

RATE LIMITS

- Upstream: **120 req/min per IP, 600 req/min per key**, shared bucket across analytics.
- Emma self-throttles below that: a cross-instance token bucket (~500/min via the `consume_rate_token` RPC), single-flight locks per cache key, and stale-while-revalidate caching (§7.3), so bursts of dashboard traffic never hit upstream 1:1.
- `oliviaFetch` retries 429s up to 2× honoring `Retry-After` (capped 30s + jitter). Voice-bridge starts use `maxRetries: 0` so concurrency 429s surface instantly.

PII (VERY IMPORTANT)

- List & drill-down endpoints **omit PII fields entirely** unless the key carries `dashboard:pii` — build for the redacted shape by default.
- Never returned under any scope: raw provider cost, per-lead/per-call cost, internal notes-intelligence, custom fields, CRM ids, provider call ids.
- Spend figures are **billed** amounts (what the agency is charged), basis `billed_voice`, integer cents.

Analytics API

One discovery endpoint plus sixteen per-client endpoints under `/api/external/v1/clients/{clientId}/...` All are GET (the single exception: the notes PUT) and all require `dashboard:read` unless marked otherwise.

4.1 Discovery

GET `/api/v1/external/clients`

● `clients:read`

Lists the agency's clients — run once, cache, refresh periodically. Each `clients[].id` is the `{clientId}` for every endpoint below. Emma mirrors this roster into Supabase (`olivia_clients`, `resync ≤ every 10 min` or forced from the console).

PARAMS `page`, `limit` ($≤ 100$)

RETURNS `clients[] { id, name, slug, status, industry, website, timezone, created_at } + total, page, limit`

4.2 Aggregates & performance

GET `/clients/{clientId}/overview`

● `dashboard:read`

Headline KPIs for the window — powers the hero cards and “Brief Emma” agenda.

```
{
  "client_id": "8f3b...",
  "period": { "from": "2026-05-21", "to": "2026-06-20", "tz": "America/New_York" },
  "kpis": {
    "leads_total": 120, "calls_total": 64, "pickup_rate": 0.52,
    "avg_call_duration_sec": 138, "bookings_rate": 0.31, "converted_count": 12,
    "spend": { "total_cents": 84210, "currency": "usd", "basis": "billed_voice" },
    "leads_by_stage": { "new": 40, "contacted": 30, "qualified": 18, "booked": 20,
      "converted": 12, "lost": 0, "dnc": 0 }
  }
}
```

GET `/clients/{clientId}/timeseries`

● `dashboard:read`

Daily series for trend charts — one zero-filled entry per tz-local calendar day.

PARAMS `from`, `to`, `tz`

RETURNS `series[] { date, calls, picked_up, bookings, spend_cents }`

GET `/clients/{clientId}/outcomes`

● `dashboard:read`

Distribution maps for the Outcomes donuts/bars. Keys are dynamic — only non-zero buckets appear (full enum sets in S8).

RETURNS `outcomes { call_outcomes, call_dispositions, booking_outcomes } — each a { value: count } map`

GET `/clients/{clientId}/funnel`[● dashboard:read](#)

Lead-stage counts for the window — same seven stages as `leads_by_stage`.

RETURNS `funnel { new, contacted, qualified, booked, converted, lost, dnc }`

GET `/clients/{clientId}/agents`[● dashboard:read](#)

Per-agent leaderboard — one row per active outbound setting agent.

RETURNS `agents[] { agent_id, name, client_name, total_leads, total_calls, pickup_rate, overall_booking_rate, call_to_booking_rate, avg_call_duration_sec, total_conversion }`

CAVEAT Lead-denominated columns (`total_leads`, `overall_booking_rate`, `total_conversion`) are client-level; only call-denominated columns are agent-scoped. `total_conversion` = `overall_booking_rate`. No cost field.

GET `/clients/{clientId}/campaigns`[● dashboard:read](#)

Reactivation-campaign performance. Counts are **lifetime-to-date** — not sliced by `from` / `to`. No PII, no cost.

RETURNS `campaigns[] { id, name, status, created_at, leads_total, leads_contacted, replies, opt_outs, appointments_booked, reply_rate, opt_out_rate } + total`

GET `/clients/{clientId}/pipelines`[● dashboard:read](#)

Pipeline & stage definitions that drive the Pipeline board (columns, ordering, stage colors). Stage ids feed `/leads?stage_id=...` for per-column pagination.

RETURNS `pipelines[]` with their ordered `stages[]`

GET `/clients/{clientId}/calendar`[● dashboard:read](#)

Booking events for the month grid plus the upcoming list. Dates/times arrive already localized to the client's timezone.

PARAMS `month ( YYYY-MM ), upcoming_limit`

RETURNS `CalendarResponse` — events grouped for the grid + upcoming bookings

4.3 Records & drill-downs

GET `/clients/{clientId}/leads`[● dashboard:read](#)[● + pii](#)

Paginated lead list (the Leads table and each board column via `stage_id`).

PARAMS `from, to, page, limit` + filters `status, source, stage_id`

BASE `leads[] { id, status, source, tags, industry, timezone, created_at, updated_at, last_call_at, last_disposition, total_calls, stage_entered_at }`

WITH PII adds `first_name, last_name, email, phone, instagram_handle`

NEVER per-lead cost, internal notes-intelligence, custom fields, external/CRM ids

GET `/clients/{clientId}/leads/{leadId}`

● dashboard:read

Single-lead detail for the lead page (profile, activity, notes). 404 `lead_not_found` for foreign/missing ids.

PUT `/clients/{clientId}/leads/{leadId}/notes`

● dashboard:notes

The only write in the analytics tree: persists a lead's notes (empty string clears, max 20,000 chars). A key without the scope gets 403 and Emma renders notes read-only. Sent with `maxRetries: 0`.

BODY `{ "notes": "..." }`

RETURNS `{ notes, updated_at }`

GET `/clients/{clientId}/calls`

● dashboard:read

● + pii

Paginated call log. Only real lead calls (internal/tester calls excluded).

BASE `calls[] { id, lead_id, direction, status, disposition, started_at, ended_at, duration_seconds, created_at }`

WITH PII adds `from_number, to_number, recording_url, transcript, callback_notes`

NEVER `per-call cost, provider call id`

GET `/clients/{clientId}/conversations`

● dashboard:read

● + pii

Paginated conversation log (SMS / DM / email threads). Internal reporting & tester threads excluded; message bodies are *not* here — use the thread endpoint.

BASE `conversations[] { id, lead_id, channel, platform, started_at, ended_at, sentiment_score, opted_out_at, created_at }`

WITH PII adds `summary` (AI recap)

GET `/clients/{clientId}/conversations/{conversationId}`

● dashboard:pii

Full message-by-message thread for the chat drawer (default 100, max 500 messages). Without `dashboard:pii` the thread returns `locked: true` with `messages: []` — the drawer shows a locked state instead of bodies.

GET `/clients/{clientId}/dm-threads`

● dashboard:read

Paginated DM thread index (Instagram / WhatsApp etc. ride on `platform`). Filterable by `lead_id`; bodies live behind the thread endpoint above.

4.4 Agency level

GET `/agencies/{agencyId}/clients`

● dashboard:read

Per-client billed cost roll-up for the console's Costs view. `{agencyId}` must be the key's own agency (else 404 `agency_not_found`) — Emma supplies it via `OLIVIA_AGENCY_ID`; the view is disabled when unset. Money is integer cents; server `totals` are used verbatim, never recomputed.

PARAMS `range` (e.g. `30d`) or `from, to, tz`; optional `client_id`

4.5 Endpoint → dashboard section

DASHBOARD SECTION	ENDPOINT(S)
Overview — hero KPI cards, Brief Emma agenda	/overview (+ /campaigns)
Pipeline — board columns & cards	/pipelines + /leads?stage_id
Calendar — month grid & upcoming	/calendar
Outcomes — distributions, trends	/outcomes + /timeseries
Agents — leaderboard & drill-down	/agents
Leads — table & lead page	/leads · /leads/{id} · PUT /leads/{id}/notes
Calls & Conversations — log, drawers, recordings	/calls · /conversations · /conversations/{id} · /dm-threads
Console — roster, costs	/api/v1/external/clients · /agencies/{id}/clients

Voice bridges

The two places the API is not a plain GET: starting an ephemeral **voice session** where Emma narrates a client's numbers over a browser call (Retell web call). Actions never mutate leads, calls or analytics.

SESSION LIFECYCLE  →  →  →  / 

5.1 Briefing bridge

“Brief Emma” on the Overview: Emma voice-narrates the catch-up agenda (bookings to confirm → new leads to work → leads to chase → conversions → campaign results) for a window and focus.

POST /clients/{clientId}/briefings

Start a briefing. Returns the prioritized agenda plus short-lived Retell web-call credentials, bound to this `briefing_id` + `client_id`. With `voice: false`, returns the agenda only. Concurrency-capped (≤ 3 live per key) → 429 + Retry-After.

BODY { from?, to?, tz?, focus?: "all"|"bookings"|"leads"|"campaigns", voice?: bool }

```
{
  "briefing_id": "brf_8f3b...", "client_id": "8f3b...", "status": "connecting",
  "agenda": [ { "id": "bookings", "category": "bookings",
    "title": "12 appointments to confirm", "priority": 1 }, ... ],
  "realtime": { "provider": "retell", "access_token": "eyJ...", "call_id": "call_abc123", "expires_at": "..." }
}
```

GET /clients/{clientId}/briefings/{briefingId}

Status + running transcript, with `current_item_id` tracking which agenda item Emma is speaking (drives the “Speaking now” highlight; poll ~2s).

POST /clients/{clientId}/briefings/{briefingId}/end

Ends the briefing and tears down the voice session → { "status": "ended" }.

5.2 Reporting bridge

“Reporting walkthrough” next to Brief Emma: headline metrics, today’s call schedule and outstanding looms, with optional voice drill-down into one agent. Strictly single-client; cross-client comparison is disabled server-side. Optional `dashboard:pii` lets Emma speak lead names.

METHOD	PATH (UNDER .../CLIENTS/{CLIENTID})	PURPOSE
POST	/reporting	Start a session. Body { from?, to?, tz?, agent_id? } → 201 { reporting_id, status, period, summary { schedule_count, loom_count }, realtime } .Join Retell within ~60s. Limit 3 concurrent/agency → 429 reporting_concurrency_limit + Retry-After: 30 .
GET	/reporting/{id}	Status + transcript (polling fallback ~2s): { status, started_at, ended_at, transcript[] { role, text, at } } .
GET	/reporting/{id}/stream	SSE (preferred): named events status / transcript / ended .Closes on terminal status or ~110s.
POST	/reporting/{id}/end	Idempotent end → { "status": "ended" } . Called on hang-up / unmount.

Activation. Both bridges are flag-gated in Emma: OLIVIA_BRIEFING_ENABLED / OLIVIA_REPORTING_ENABLED. Until the flag is on *and* the key carries the scope, the modals run a local simulated preview — same UI, no live call. The browser only ever receives the short-lived realtime join token, never the agency key.

Dashboard HTTP endpoints

Emma's own Next.js route handlers — same-origin endpoints the browser (or Supabase / Vercel cron) calls. Each one self-authenticates; middleware deliberately excludes `/api/*`.

GET `/api/reporting/{id}`

● Supabase session

Reporting-status proxy (polling fallback). The `client_id` is resolved from the session server-side, so a user can only read their own workspace's session. `Cache-Control: private, no-store`; upstream errors pass through (or `502`) with `Retry-After` preserved.

GET `/api/reporting/{id}/stream`

● Supabase session

Same-origin **SSE proxy** for the live reporting transcript. Opens the upstream stream server-side (browser `EventSource` can't send the key) and pipes `status` / `transcript` / `ended` events verbatim. Browser abort flows through `request.signal` to tear down upstream. `maxDuration: 120s`.

GET `/api/calls/{id}/recording?src=...`

● Supabase session

Call-recording download proxy (recordings are cross-origin without CORS). **SSRF-guarded**: `src` must be https and host-allowlisted (`RECORDING_HOST_ALLOWLIST`, default = the voice provider's CDN). Streams as `attachment`; `filename="call-<id>.mp3"`. Errors: `400 bad_src`, `502 upstream_*`.

POST `/api/auth/email-hook`

● webhook signature

Supabase "Send Email" auth hook. Verified with **Standard Webhooks** signatures (`SEND_EMAIL_HOOK_SECRET`); invalid signature → `401`. Renders the branded magic-link template and sends via Resend; notification-only payloads (no `token_hash`) are acknowledged without sending.

GET `/api/cron/snapshot`

● cron bearer secret

Daily snapshot endpoint (Vercel cron `5 0 * * *`). **Disabled by default** (`SNAPSHOTS_ENABLED`); when enabled it is fail-closed — `CRON_SECRET` must be set (`500` if not) and the request must carry `Authorization: Bearer <CRON_SECRET>` (`401` otherwise). Runs `runDailySnapshots()` across all mirrored clients.

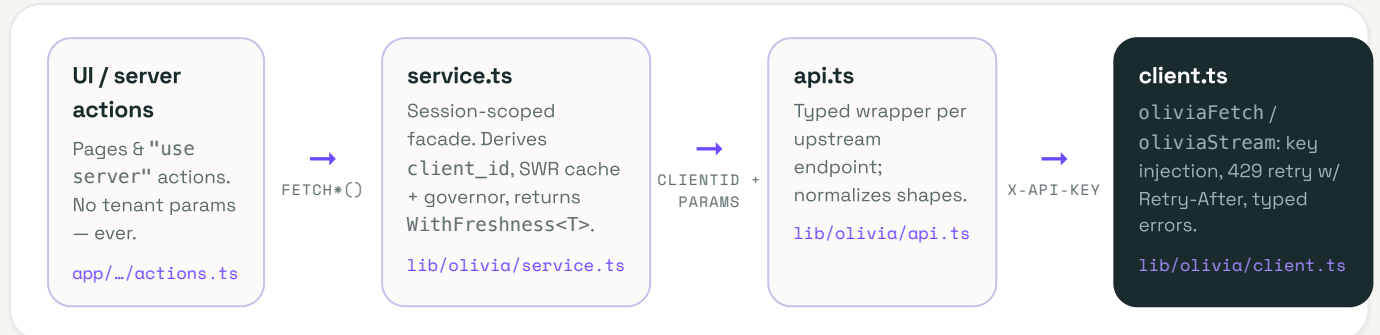
GET `/auth/callback`

● magic-link token

Magic-link landing: exchanges `?code` (PKCE) or `?token_hash&type` for a Supabase session, then `302` to `next` (default `/dashboard`); failures bounce to `/login?error=link`.

Server function layer

How a page gets its data: UI → server actions / server components → `service.ts` (session-scoped, cached) → `api.ts` (typed endpoints) → `client.ts` (key injection, retries). Pages never call `api.ts` directly.



7.1 Typed endpoint wrappers — `lib/olivia/api.ts`

FUNCTION	UPSTREAM ENDPOINT	SCOPE
<code>discoverClients</code>	GET <code>/api/v1/external/clients</code>	<code>clients:read</code>
<code>getOverview</code>	GET <code>/clients/{id}/overview</code>	<code>dashboard:read</code>
<code>getTimeSeries</code>	GET <code>/clients/{id}/timeseries</code>	<code>dashboard:read</code>
<code>getOutcomes</code>	GET <code>/clients/{id}/outcomes</code>	<code>dashboard:read</code>
<code>getFunnel</code>	GET <code>/clients/{id}/funnel</code>	<code>dashboard:read</code>
<code>getPipelines</code>	GET <code>/clients/{id}/pipelines</code>	<code>dashboard:read</code>
<code>getAgents</code>	GET <code>/clients/{id}/agents</code>	<code>dashboard:read</code>
<code>getCampaigns</code>	GET <code>/clients/{id}/campaigns</code>	<code>dashboard:read</code>
<code>getLeads</code>	GET <code>/clients/{id}/leads</code>	<code>dashboard:read</code>
<code>getLeadDetail</code>	GET <code>/clients/{id}/leads/{leadId}</code>	<code>dashboard:read</code>
<code>getLeadDirectory</code>	pages <code>/leads</code> (365d, ≤25 pages) → <code>id→name</code> map	<code>read + pii</code>
<code>putLeadNotes</code>	PUT <code>/clients/{id}/leads/{leadId}/notes</code>	<code>dashboard:notes</code>
<code>getCalls</code>	GET <code>/clients/{id}/calls</code>	<code>dashboard:read</code>
<code>getConversations</code>	GET <code>/clients/{id}/conversations</code>	<code>dashboard:read</code>
<code>getConversationThread</code>	GET <code>/clients/{id}/conversations/{convId}</code>	<code>dashboard:pii</code>
<code>getDmThreads</code>	GET <code>/clients/{id}/dm-threads</code>	<code>dashboard:read</code>
<code>getCalendar</code>	GET <code>/clients/{id}/calendar</code>	<code>dashboard:read</code>
<code>getAgencyClientCosts</code>	GET <code>/agencies/{agencyId}/clients</code>	<code>dashboard:read</code>
<code>startBriefing / endBriefing</code>	POST <code>/clients/{id}/briefings[{bid}/end]</code>	<code>dashboard:brief</code>
<code>startReporting / getReportingStatus / endReporting</code>	POST/GET <code>/clients/{id}/reporting...</code>	<code>dashboard:report</code>

FUNCTION	UPSTREAM ENDPOINT	SCOPE
streamReporting	GET /clients/{id}/reporting/{rid}/stream (SSE)	dashboard:report

7.2 Session-scoped facade — lib/olivia/service.ts

The only surface pages and actions may use. Every export first calls `getSessionClientId()`, then routes through the cache. Reads return `WithFreshness<T> ( { data, fetchedAt, stale } )` so the UI can show a freshness hint.

READS	<code>fetchOverview · fetchTimeseries · fetchOutcomes · fetchFunnel · fetchAgents · fetchCampaigns · fetchPipelines · fetchLeads · fetchCalls · fetchConversations · fetchCalendar · fetchLeadDetail · fetchDmThreads · fetchConversationThread</code>
WRITES	<code>saveLeadNotes</code> (uncached PUT, then force-refreshes the cached lead detail)
VOICE	<code>startBriefing · endBriefing · startReporting · reportingStatus · streamReporting · endReporting</code> — flag-gated; degrade to <code>{ mode: "simulated" }</code>
JOINS	<code>fetchCalls</code> / <code>fetchConversations</code> best-effort join lead names via the PII lead directory

7.3 Cache & governor — lib/olivia/cache.ts, governor.ts

STALE-WHILE-REVALIDATE

- Backed by the Supabase `response_cache` table; per-endpoint freshness tiers (overview 60s/300s ... threads 15s/60s ... discovery 1h/24h).
- Serves fresh → stale → last-known-good, refreshing in the background; single-flight lock prevents thundering herds.
- Cache keys always embed `client_id` — tenant isolation by construction.

CROSS-INSTANCE RATE GOVERNOR

- Postgres RPCs: `consume_rate_token` (token bucket ~500/min, under the upstream 600/min), `try_acquire_lock`, `release_lock`.
- Fail-open on infra errors; over budget with no cache → typed `429 rate_limited` to the UI.

7.4 Server actions — app/.../actions.ts

ACTION	AUTH	DOES
<code>sendMagicLink(email)</code>	public	Passwordless sign-in via OTP email; never auto-creates users (<code>otp_disabled</code> → "no account").
<code>passwordSignIn(email, pw)</code>	public	Password fallback; redirects to <code>/dashboard</code> .
<code>signOut()</code>	session	Signs out, redirects to <code>/login</code> .
<code>setActiveClient(clientId)</code>	platform_admin	Switches the admin's active workspace (validated against <code>olivia_clients</code> ; sets the <code>httpOnly</code> cookie).
<code>fetchBriefWindow(window)</code>	session	Recomputes the Brief Emma agenda for a date window (overview + campaigns).
<code>beginBrief / endBrief</code>	session	Start/end a briefing session (§5.1) — returns live realtime creds or simulated mode.
<code>beginReport / endReport</code>	session	Start/end a reporting walkthrough (§5.2), optional <code>agent_id</code> drill-down.
<code>loadThread(conversationId)</code>	session	Loads a DM/SMS thread for the chat drawer; 404s map to <code>not_found</code> , never retried.

ACTION	AUTH	DOES
<code>saveNotes(leadId, notes)</code>	session	Persists lead notes (§4.3); <code>forbidden_scope</code> → read-only UI state.
<code>loadStageLeads(stageId, page)</code>	session	One page of pipeline-board cards for a stage column.
<code>loadPipelineStages(pipelineId)</code>	session	Lazily prefetches a pipeline's stage columns.
<code>refreshBoard(pipelineId)</code>	session	Force-refreshes the board past the SWR fresh window.
<code>createInvite(clientId, email)</code>	platform_admin	Single-use 7-day workspace invite (role <code>member</code>) + branded email.
<code>revokeInvite(id)</code>	platform_admin	Revokes a pending invite.
<code>createTeamInvite(email)</code>	platform_admin	Invites an agency teammate as <code>platform_admin</code> .
<code>syncClients()</code>	platform_admin	Force-resyncs the client roster from discovery.
<code>acceptInvite(token)</code>	invite token	Validates the token, provisions the Supabase user, maps membership (refusing conflicts), sends a magic link.

7.5 Console data functions — `lib/olivia/agency.ts` • `requireAdmin()`

ROSTER	<code>listAgencyClients</code> · <code>getAgencyClient</code> · <code>refreshAgencyClients</code> · <code>getMirrorSyncedAt</code> — the Supabase mirror of discovery, joined with member counts
OVERVIEW	<code>getAgencyOverview</code> — aggregates every client's overview + outcomes into agency totals and a leaderboard
COSTS	<code>getAgencyCosts</code> — per-client billed cost via §4.4 (needs <code>OLIVIA_AGENCY_ID</code> ; view disabled when unset)
PEOPLE	<code>listInvites</code> · <code>listMembers</code> · <code>listAgencyTeam</code> · <code>getClientDetail</code> — invites + memberships with emails resolved via Supabase admin

7.6 Pure helpers (no I/O)

`lib/overview.ts` (KPI cards, Brief agenda) · `lib/pipeline/board.ts` (board math) · `lib/calendar.ts` (month grid) · `lib/best-times.ts` (heatmap) · `lib/dm.ts` (channel labels) · `lib/recording.ts` (SSRF guard) — transformers over already-fetched data; they never touch the network.

Enums & metric definitions

The exact value sets and metric semantics — rendered here in the same badge colors the dashboard uses (`lib/design.ts`).

LEAD STATUS / FUNNEL STAGES

● New ● Contacted ● Qualified ● Booked
● Converted ● Lost ● DNC

CALL STATUS

● Queued ● Ringing ● In progress
● Completed ● Failed ● No answer ● Busy

Legacy `voicemail` folds into `no_answer`.

CALL DISPOSITION

● Interested ● Not interested
● Callback requested ● Wrong number
● Voicemail left ● Booked ● DNC
● No disposition

BOOKING STATUS

● Scheduled ● Confirmed ● Completed
● Cancelled ● No show

CAMPAIGN STATUS

● Draft ● Active ● Paused ● Completed
● Archived

LEAD SOURCE · CONVERSATION CHANNEL

csv_import crm_sync manual api webhook
sms_unknown direct_booking
reactivation_campaign cliniko_sync
hubspot_sync
voice sms email chat imessage
platform: instagram · whatsapp · ...

8.1 Metric definitions (don't guess — exact semantics)

METRIC	DEFINITION
<code>bookings_rate</code>	The “conversion” KPI: distinct in-window leads with a non-cancelled booking or <code>status = 'booked'</code> ÷ total in-window leads. Excludes the manual won-deal <code>converted</code> status.
<code>converted_count</code>	Leads in the <code>converted</code> stage (manual “won deal”) — deliberately shown separately from bookings.
<code>calls_total</code>	Distinct leads with a <i>real conversation</i> (completed call > 10s, or in progress) — not raw call rows. Raw per-day counts live in <code>/timeseries</code> and <code>/outcomes</code> .
<code>pickup_rate</code>	Outbound only, lead-level: leads that ever picked up ÷ leads attempted (inbound excluded).
<code>avg_call_duration_sec</code>	Mean duration over completed conversations only.
<code>spend.total_cents</code>	Billed voice spend for the window (<code>basis: "billed_voice"</code>) — what the agency is charged, never the provider's raw cost. No per-category LLM/SMS split; TTS is bundled into voice.
<code>campaign counts</code>	Lifetime-to-date (never period-sliced).

Environment & security

Every credential and flag the API surface depends on — names and purpose only; values live in Vercel env / a secret store.

VARIABLE	KIND	PURPOSE
OLIVIA_API_KEY	secret	The agency API key (§2.1). Server-only; sent as <code>x-api-key</code> on every upstream call.
OLIVIA_API_BASE	config	Upstream host override (default <code>https://www.lunarolivia.com</code>).
OLIVIA_AGENCY_ID	config	Agency id for the costs endpoint (§4.4); costs view disabled when unset.
OLIVIA_BRIEFING_ENABLED	flag	Enables the live briefing bridge (else simulated preview).
OLIVIA_REPORTING_ENABLED	flag	Enables the live reporting bridge (else simulated preview).
NEXT_PUBLIC_SUPABASE_URL · _ANON_KEY	public	Supabase project + anon key (RLS-enforced user client).
SUPABASE_SERVICE_ROLE_KEY	secret	Service-role client (bypasses RLS): provisioning, governor RPCs, cache, snapshots. Server-only.
SEND_EMAIL_HOOK_SECRET	secret	Standard Webhooks signing secret for <code>/api/auth/email-hook</code> .
CRON_SECRET	secret	Bearer secret for <code>/api/cron/snapshot</code> (fail-closed when snapshots are enabled).
SNAPSHOTS_ENABLED	flag	Activates the daily snapshot cron (Phase 8; off by default).
RESEND_API_KEY	secret	Resend key for transactional email.
EMAIL_REPLY_TO	config	Reply-to for outbound email.
RECORDING_HOST_ALLOWLIST	config	Comma-separated allowlist for the recording proxy's SSRF guard (default: the voice provider's CDN host).

9.1 Security checklist

- The agency key lives in a backend secret store and never reaches the browser — all upstream calls are server-side; anything the frontend needs is proxied same-origin.
- Request `dashboard:pii` only where the product genuinely needs names/contacts/transcripts; the UI is built for the redacted shape by default.
- Every `client_id` comes from the session. A wrong or foreign id returns `404 client_not_found` — never data.
- Handle `401` (key rotated/revoked), `403` (missing scope), `429` (back off with `Retry-After`), `400` (fix params) — all typed as `OliviaError`.
- Voice-bridge tokens are short-lived, single-session, and bound to one `briefing_id` / `reporting_id` + `client_id`; ending a session tears down the call.
- Webhooks and crons authenticate independently: Standard Webhooks signatures for the email hook, a fail-closed bearer secret for the snapshot cron, an SSRF host-allowlist for the recording proxy.